

Order Management System

Database Management System — Week 4 Mini Project

Task	Order Management System
Subject	Database Management System (DBMS)
Week	Week 4
Core Tables	Orders, Order_Details

1. Objective

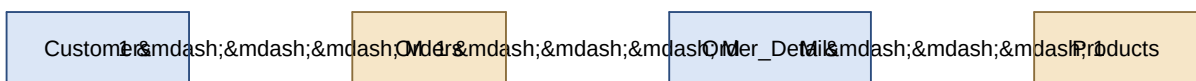
The goal of this task is to design and implement a small-scale **Order Management System** using a relational database. The system models how customers place orders for products, how each order's line items are stored, and how order records can be inserted, updated, and queried to produce a customer's order history.

2. Scope of the Task

- Design the **Orders** and **Order_Details** tables (with supporting Customers and Products tables).
- Manage customer product orders — one order can contain multiple products.
- Store order date, quantity ordered, and total amount for each order/line item.
- Perform order insertion (new order) and modification (update quantity, status, etc.) operations.
- Generate a customer order history report using JOIN and aggregate queries.

3. Entity-Relationship Concept

A single order can include many different products, and a single product can appear in many different orders — this is a classic **many-to-many** relationship between Orders and Products. It is resolved using a junction (bridge) table, **Order_Details**, which stores one row per product within an order.



4. Table Design

4.1 Customers Table

4.2 Products Table

4.3 Orders Table (core)

4.4 Order_Details Table (core)

Column	Data Type	Constraint
customer_id	INT	PRIMARY KEY, AUTO_INCREMENT
customer_name	VARCHAR(100)	NOT NULL
email	VARCHAR(100)	UNIQUE
phone	VARCHAR(15)	—
address	VARCHAR(200)	—

4.2 Products Table

Column	Data Type	Constraint
product_id	INT	PRIMARY KEY, AUTO_INCREMENT
product_name	VARCHAR(100)	NOT NULL
price	DECIMAL(10,2)	NOT NULL, CHECK (price >= 0)
stock_qty	INT	DEFAULT 0

4.3 Orders Table (core)

Column	Data Type	Constraint
order_id	INT	PRIMARY KEY, AUTO_INCREMENT
customer_id	INT	FOREIGN KEY REFERENCES Customers(customer_id)
order_date	DATE	NOT NULL, DEFAULT CURRENT_DATE
total_amount	DECIMAL(10,2)	NOT NULL DEFAULT 0.00
order_status	VARCHAR(20)	DEFAULT 'Pending'

4.4 Order_Details Table (core)

Column	Data Type	Constraint
order_detail_id	INT	PRIMARY KEY, AUTO_INCREMENT
order_id	INT	FOREIGN KEY REFERENCES Orders(order_id)
product_id	INT	FOREIGN KEY REFERENCES Products(product_id)
quantity	INT	NOT NULL, CHECK (quantity > 0)
unit_price	DECIMAL(10,2)	NOT NULL
line_total	DECIMAL(10,2)	quantity * unit_price

5. SQL — Creating the Tables

```
CREATE TABLE Customers (  
    customer_id    INT AUTO_INCREMENT PRIMARY KEY,  
    customer_name  VARCHAR(100) NOT NULL,  
    email          VARCHAR(100) UNIQUE,  
    phone          VARCHAR(15),  
    address        VARCHAR(200)  
);  
  
CREATE TABLE Products (  
    product_id     INT AUTO_INCREMENT PRIMARY KEY,  
    product_name   VARCHAR(100) NOT NULL,  
    price          DECIMAL(10,2) NOT NULL CHECK (price >= 0),  
    stock_qty      INT DEFAULT 0  
);  
  
CREATE TABLE Orders (  
    order_id       INT AUTO_INCREMENT PRIMARY KEY,  
    customer_id    INT NOT NULL,  
    order_date     DATE NOT NULL DEFAULT (CURRENT_DATE),  
    total_amount   DECIMAL(10,2) NOT NULL DEFAULT 0.00,  
    order_status   VARCHAR(20) DEFAULT 'Pending',  
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)  
);  
  
CREATE TABLE Order_Details (  
    order_detail_id INT AUTO_INCREMENT PRIMARY KEY,  
    order_id        INT NOT NULL,  
    product_id      INT NOT NULL,  
    quantity        INT NOT NULL CHECK (quantity > 0),  
    unit_price       DECIMAL(10,2) NOT NULL,  
    line_total       DECIMAL(10,2) GENERATED ALWAYS AS (quantity * unit_price) STORED,  
    FOREIGN KEY (order_id) REFERENCES Orders(order_id) ON DELETE CASCADE,  
    FOREIGN KEY (product_id) REFERENCES Products(product_id)  
);
```

6. Order Insertion Operations

Placing a new order involves two steps: creating the order header in **Orders**, then adding each purchased product as a row in **Order_Details**.

```
-- Step 1: Insert a new customer (if not already existing)  
INSERT INTO Customers (customer_name, email, phone, address)  
VALUES ('Ravi Kumar', 'ravi.kumar@mail.com', '9876543210', 'Chennai, TN');  
  
-- Step 2: Create the order header  
INSERT INTO Orders (customer_id, order_date, order_status)  
VALUES (1, CURRENT_DATE, 'Pending');  
  
-- Step 3: Add line items to Order_Details  
INSERT INTO Order_Details (order_id, product_id, quantity, unit_price)  
VALUES (1, 101, 2, 499.00),  
      (1, 105, 1, 1299.00);  
  
-- Step 4: Update the order's total_amount from its line items  
UPDATE Orders
```

```

SET total_amount = (
    SELECT SUM(line_total) FROM Order_Details WHERE order_id = 1
)
WHERE order_id = 1;

```

7. Order Modification Operations

```

-- Update quantity of a product in an existing order
UPDATE Order_Details
SET quantity = 3
WHERE order_id = 1 AND product_id = 101;

-- Recalculate total_amount after modification
UPDATE Orders
SET total_amount = (
    SELECT SUM(line_total) FROM Order_Details WHERE order_id = 1
)
WHERE order_id = 1;

-- Update order status (e.g., Pending -> Shipped)
UPDATE Orders
SET order_status = 'Shipped'
WHERE order_id = 1;

-- Cancel / remove one product line from an order
DELETE FROM Order_Details
WHERE order_id = 1 AND product_id = 105;

```

8. Customer Order History Report

Order history is generated by joining **Customers** → **Orders** → **Order_Details** → **Products**, so each row shows which customer bought what, when, in what quantity, and for how much.

```

-- Detailed order history for one customer
SELECT
    c.customer_name,
    o.order_id,
    o.order_date,
    p.product_name,
    od.quantity,
    od.unit_price,
    od.line_total,
    o.order_status
FROM Customers c
JOIN Orders o      ON c.customer_id = o.customer_id
JOIN Order_Details od ON o.order_id   = od.order_id
JOIN Products p     ON od.product_id = p.product_id
WHERE c.customer_id = 1
ORDER BY o.order_date DESC;

-- Summary: total orders and total amount spent per customer
SELECT
    c.customer_name,
    COUNT(DISTINCT o.order_id) AS total_orders,
    SUM(od.line_total)         AS total_spent
FROM Customers c

```

```
JOIN Orders o          ON c.customer_id = o.customer_id
JOIN Order_Details od ON o.order_id    = od.order_id
GROUP BY c.customer_name
ORDER BY total_spent DESC;
```

Sample Output

Customer	Order ID	Date	Product	Qty	Unit Price	Line Total
Ravi Kumar	1	2026-08-10	Wireless Mouse	3	499.00	1497.00
Ravi Kumar	2	2026-08-15	Bluetooth Speaker	1	1299.00	1299.00

9. Conclusion

This task demonstrates the core relational-design skills covered in Week 4: normalizing a many-to-many relationship using a junction table (Order_Details), enforcing data integrity with primary/foreign keys and CHECK constraints, performing INSERT/UPDATE/DELETE operations to manage live order data, and using multi-table JOIN queries with aggregation to build a meaningful business report (customer order history).